

CAPESTONE: MySQL Analytics

10

Advanced SQL Queries

Window functions,CTEs,aggregations&Pearson correlation

5

Fiscal Years

Multi-year financialtrendanalysisacrossthe dataset

448

Companies spanning11GICSsectors

11

Full cross-sectorfinancialcoverage



Table of Contents: 10 Advanced SQL Queries

Aggregation & Filtering (Q1–Q4)

- **Q1** — Top 3 Companies by Avg Net Income · AVG · GROUP BY · LIMIT
- **Q2** — YoY Revenue Growth for AAL · LAG · CTE · NULLIF
- **Q3** — Negative Profit Margin Companies · Computed Ratio
- **Q4** — Avg Quick Ratio Below 1 · HAVING · Normalization

Window Functions & Advanced Analytics (Q5–Q10)

- **Q5** — 3-Year Rolling Avg R&D · ROWS BETWEEN · Window Frame
- **Q6** — 3+ Consecutive Rising CapEx · Double LAG · CTE
- **Q7** — Top 5 Years by Total Liabilities · SUM · ORDER BY
- **Q8** — Max Operating Cash Flow per Company · RANK · CTE
- **Q9** — High-Equity High-Margin by EPS · CROSS JOIN · CTE
- **Q10** — R&D vs Net Income Pearson r · Pure SQL Stats

Top 3 Companies by Average Net Income

Aggregates net income across all available fiscal years per ticker and returns the top 3 companies by average. `AVG()` computes the mean across all years before ranking — `ORDER BY DESC + LIMIT 3` efficiently returns top-N results without requiring a subquery.

MySQL Query

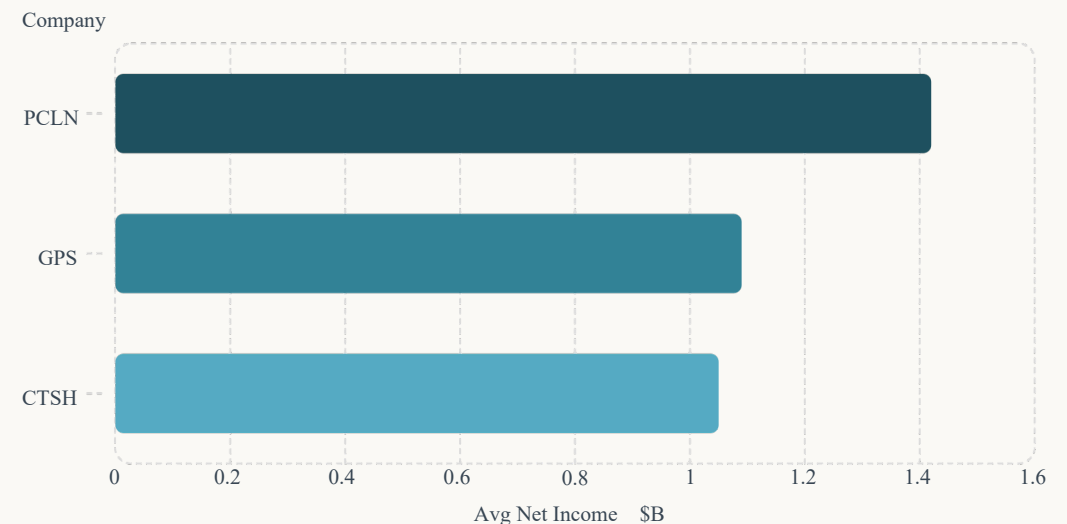
```
SELECT
  ticker_symbol,
  ROUND(AVG(net_income), 1) AS avg_net_income
FROM fundamentals
GROUP BY ticker_symbol
ORDER BY avg_net_income DESC
LIMIT 3;
```

Key Insights

- **PCLN (Priceline)** leads with **\$1.42B** average net income — driven by online travel market dominance across all fiscal years in the dataset.
- `AVG()` aggregates across all available fiscal years per company before ranking — ensuring a true multi-year profitability measure.
- **GPS** and **CTSH** both exceed \$1B average net income, demonstrating consistent multi-year profitability.
- `ORDER BY ... DESC + LIMIT 3` efficiently returns top-N without a subquery — clean and performant SQL pattern.

Results: Top 3 by Avg Net Income (USD)

Rank	Ticker	Avg Net Income
1	PCLN	\$1,419,566,000
2	GPS	\$1,091,000,000
3	CTSH	\$1,051,263,000



Year-over-Year Revenue Growth for American Airlines (AAL)

The `LAG()` window function compares each year's revenue to the prior year within a partition. A CTE separates lag computation from the final percentage calculation, improving readability. `NULLIF(prev_year_revenue, 0)` safely prevents division-by-zero errors when the prior year value is `NULL` or zero.

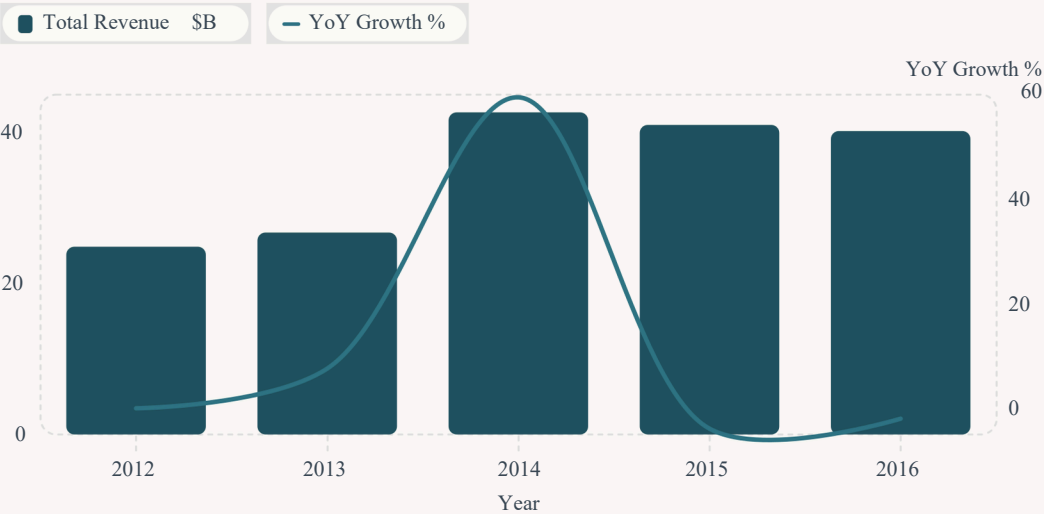
MySQL Query

```
WITH revenue_with_lag AS (  
  SELECT  
    ticker_symbol,  
    for_year,  
    total_revenue,  
    LAG(total_revenue) OVER (  
      PARTITION BY ticker_symbol  
      ORDER BY for_year  
    ) AS prev_year_revenue  
  FROM fundamentals  
  WHERE ticker_symbol = 'AAL'  
)  
SELECT  
  ticker_symbol,  
  for_year,  
  total_revenue,  
  prev_year_revenue,  
  ROUND(  
    (total_revenue - prev_year_revenue)  
    / NULLIF(prev_year_revenue, 0) * 100, 2  
  ) AS yoy_growth_pct  
FROM revenue_with_lag  
ORDER BY for_year;
```

Key Insights

- 2012 shows `NULL` for `prev_year_revenue`** — `LAG` has no preceding row, so it correctly returns `NULL` for the first year in the partition.
- 2014 shows +59.49% YoY growth** — the direct result of the US Airways merger completing in late 2013, nearly doubling AAL's revenue base.
- 2015 and 2016 show modest declines of -3.89% and -1.98% as the merged entity stabilized post-integration.
- CTE pattern: compute `LAG` first in the CTE, then apply the percentage formula in the outer `SELECT` — clean separation of concerns.

AAL Revenue & YoY Growth % (2012–2016)



Year	Total Revenue (\$)	Prev Revenue (\$)	YoY Growth %
2012	24,855,000,000	NULL	NULL
2013	26,743,000,000	24,855,000,000	+7.60%
2014	42,650,000,000	26,743,000,000	+59.49%
2015	40,990,000,000	42,650,000,000	-3.89%
2016	40,180,000,000	40,990,000,000	-1.98%

Companies with Negative Profit Margin (2012–2016)

Filters rowswhere $\text{net_income} / \text{total_revenue}$ isnegative,exposingcompanies operatingat aloss.WHERE $\text{total_revenue} <> 0$ preventsdivision-by-zero before the ratio check. ORDER BY ASC surfaces the worst performers first — a critical pattern for financial risk screening and credit analysis.

MySQL Query

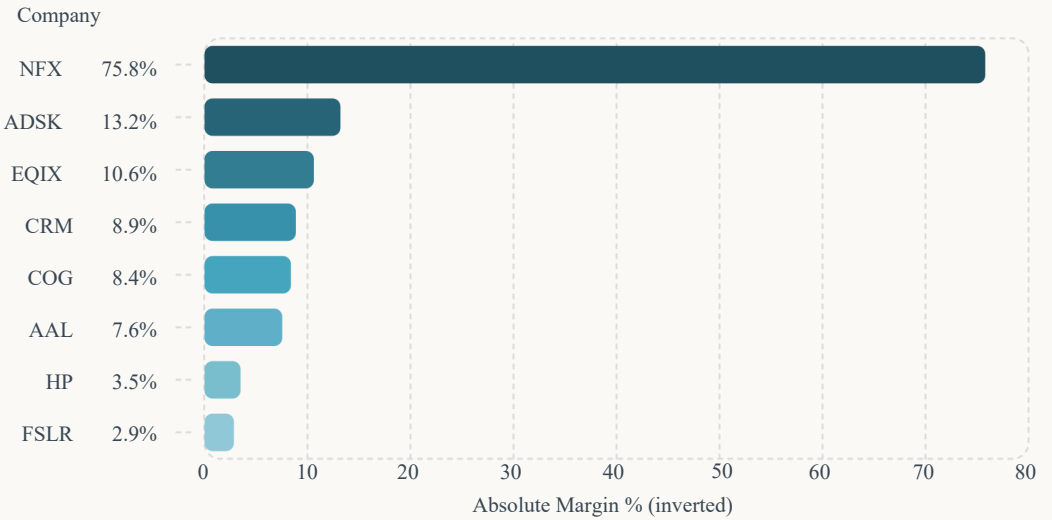
```
SELECT
  ticker_symbol,
  for_year,
  net_income,
  total_revenue,
  ROUND((net_income / total_revenue) * 100, 2) AS
profit_margin_pct
FROM fundamentals
WHERE total_revenue <> 0
  AND (net_income / total_revenue) < 0
  AND for_year BETWEEN 2012 AND 2016
ORDER BY profit_margin_pct ASC;
```

Key Insights

- **NFX worst at -75.8%** — Newfield Exploration was severely impacted by the commodity price collapse in 2012, generating only \$1.56B revenue against a \$1.18B net loss.
- **AAL 2012** had -\$1.87B net income despite \$24.8B revenue — reflecting pre-merger airline industry losses before the US Airways integration.
- WHERE $\text{total_revenue} <> 0$ is evaluated before the ratio check, preventing runtime errors on zero-revenue rows.
- ORDER BY ASC exposes worst performers first — essential for financial risk screening and distressed asset identification.

Negative Profit Margin Companies (Sorted Worst to Best)

Ticker	Year	Net Income (\$)	Revenue (\$)	Margin %
NFX	2012	-1,184,000,000	1,562,000,000	-75.80%
ADSK	2016	-330,500,000	2,504,100,000	-13.20%
EQIX	2014	-259,547,000	2,443,776,000	-10.62%
CRM	2013	-270,445,000	3,050,195,000	-8.87%
COG	2015	-113,891,000	1,357,150,000	-8.39%
AAL	2012	-1,876,000,000	24,855,000,000	-7.55%
HP	2016	-56,828,000	1,624,232,000	-3.50%
FSLR	2012	-96,338,000	3,368,545,000	-2.86%



Companies with Average Quick Ratio Below 1 — Short- Term Liquidity Risk

A quick ratio below 1 signals that current liabilities exceed liquid assets — a short-term financial stress indicator. `HAVING` filters after `GROUP BY`, since `WHERE` cannot operate on aggregate functions like `AVG()`. Raw data stores the ratio as an integer (e.g., 45 = 0.45), requiring division by 100 for normalization.

MySQL Query

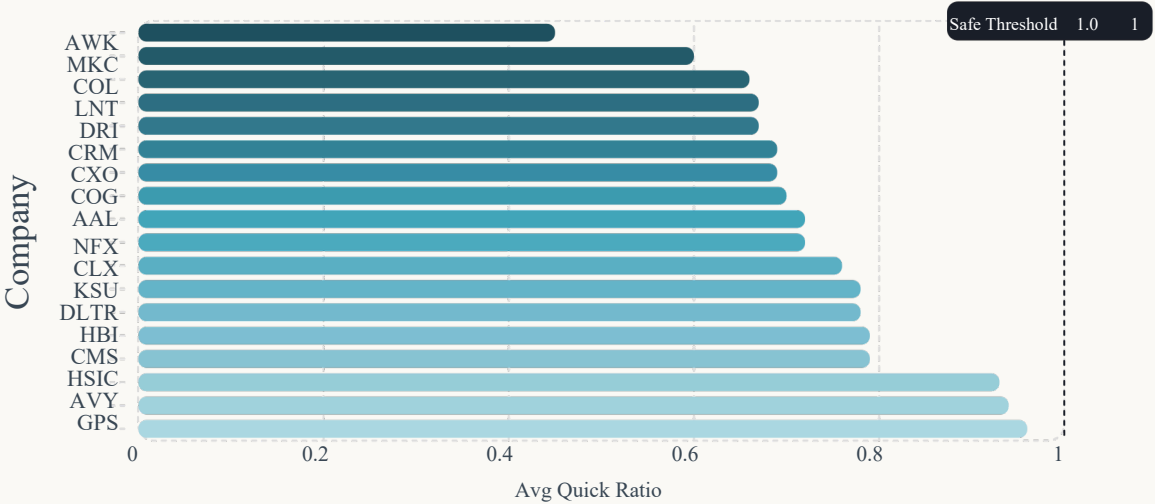
```
SELECT
  ticker_symbol,
  ROUND(AVG(quick_ratio) / 100, 2) AS
  avg_quick_ratio
FROM fundamentals
WHERE quick_ratio IS NOT NULL
  AND for_year BETWEEN 2012 AND 2016
GROUP BY ticker_symbol
HAVING AVG(quick_ratio) / 100 < 1
ORDER BY avg_quick_ratio ASC;
```

Key Insights

- **Quick ratio < 1** means current liabilities exceed liquid assets — a critical short-term risk signal for creditors and analysts.
- Raw data stores ratio as integer (e.g., 45 = 0.45), so **divide by 100** to normalize before comparison.
- **HAVING** filters after `GROUP BY` — `WHERE` cannot be used on aggregate functions like `AVG()`. This is a fundamental SQL execution order principle.
- **AWK (American Water Works)** at 0.45 represents the most severe short-term liquidity constraint in the dataset.
- 18 companies fall below the 1.0 threshold, spanning utilities, consumer staples, REITs, and healthcare sectors.

Companies with Avg Quick Ratio < 1 (Sorted by Risk Severity)

Ticker	Avg Quick Ratio	Risk Level
AWK	0.45	CRITICAL
MKC	0.60	HIGH
COL	0.66	HIGH
LNT	0.67	HIGH
DRI	0.67	HIGH
CRM	0.69	HIGH
CXO	0.69	HIGH
COG	0.70	HIGH
AAL	0.72	MEDIUM
NFX	0.72	MEDIUM
CLX	0.76	MEDIUM
KSU	0.78	MEDIUM
DLTR	0.78	MEDIUM
HBI	0.79	MEDIUM
CMS	0.79	MEDIUM
HSIC	0.93	LOW
AVY	0.94	LOW
GPS	0.96	LOW



3-Year Rolling Average of R&D Expenses

Uses `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` to create a true 3-row sliding window. `PARTITION BY ticker_symbol` resets the window independently for each company. The first row in each partition has only 1 value, so the rolling average equals the actual value itself — a natural consequence of the window frame definition.

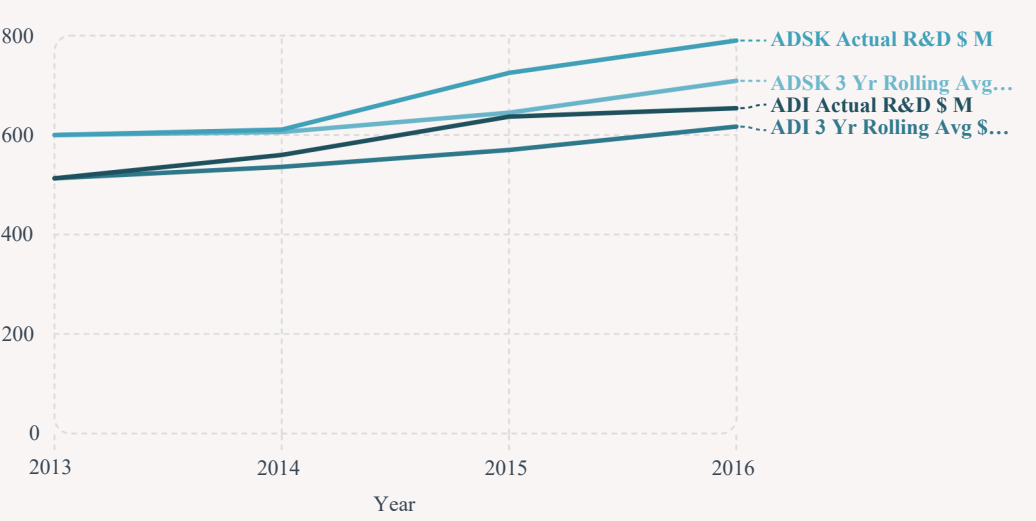
MySQL Query

```
SELECT
  ticker_symbol,
  for_year,
  research_and_development AS rd_expense,
  ROUND(
    AVG(research_and_development) OVER (
      PARTITION BY ticker_symbol
      ORDER BY for_year
      ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ), 1
  ) AS rd_3yr_rolling_avg
FROM fundamentals
WHERE research_and_development IS NOT NULL
  AND for_year BETWEEN 2013 AND 2016
ORDER BY ticker_symbol, for_year;
```

Key Insights

- ROWS BETWEEN 2 PRECEDING AND CURRENT ROW** creates a true 3- row sliding window — the current year plus the two prior years.
- First row in each partition: window has only 1 value, so rolling avg equals the actual value itself — no prior years to average with.
- PARTITION BY ticker_symbol** resets the window independently for each company, ensuring no cross-company contamination.
- ADI** R&D grew from \$513M to \$653M (2013–2016) — a **27% increase** in innovation spend over the period.
- ADSK** R&D grew from \$600M to \$790M — a 32% increase, with the 3- year rolling avg smoothing year-to-year volatility.

R&D Expenses vs 3-Year Rolling Average (ADI & ADSK)



Ticker	Year	R&D Expense (\$)	3-Yr Rolling Avg (\$)
ADI	2013	513,035,000	513,035,000
ADI	2014	559,686,000	536,360,500
ADI	2015	637,459,000	570,060,000
ADI	2016	653,816,000	616,987,000
ADSK	2013	600,000,000	600,000,000
ADSK	2014	611,100,000	605,550,000
ADSK	2015	725,200,000	645,433,333
ADSK	2016	790,000,000	708,766,667

Companies with 3+ Consecutive Years of Rising CapEx

Double LAG() detects 3-year consecutive increase patterns in capital expenditure. CapEx values are stored as negative numbers (cash outflows), so "rising" means becoming *less negative*. Both LAG(col, 1) and LAG(col, 2) are computed in a single CTE for efficiency. capex_prev2 IS NOT NULL ensures only rows with all three years of data are included.

MySQL Query

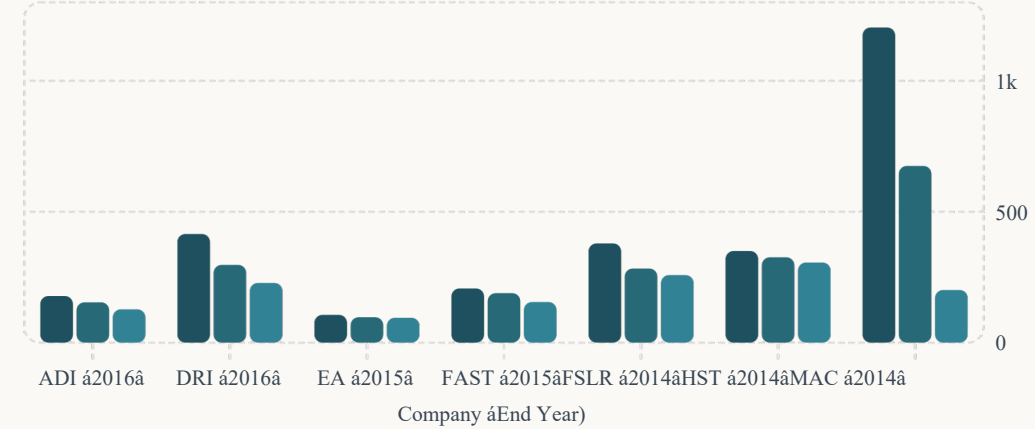
```
WITH capex_lag AS (
  SELECT
    ticker_symbol,
    for_year,
    capital_expenditures,
    LAG(capital_expenditures, 1) OVER (
      PARTITION BY ticker_symbol
      ORDER BY for_year
    ) AS capex_prev1,
    LAG(capital_expenditures, 2) OVER (
      PARTITION BY ticker_symbol
      ORDER BY for_year
    ) AS capex_prev2
  FROM fundamentals
  WHERE for_year BETWEEN 2014 AND 2016
)
SELECT
  ticker_symbol,
  for_year AS end_year,
  capex_prev2,
  capex_prev1,
  capital_expenditures AS capex_current
FROM capex_lag
WHERE capital_expenditures > capex_prev1
  AND capex_prev1 > capex_prev2
  AND capex_prev2 IS NOT NULL
ORDER BY ticker_symbol, for_year;
```

Key Insights

- CapEx values are **negative** (cash outflows) — "rising" means becoming *less negative* (smaller absolute outflow).
- LAG(col, 1) = 1 year back; LAG(col, 2) = 2 years back — both computed efficiently in a single CTE.
- capex_prev2 IS NOT NULL ensures we only include rows where all three years of data are available.
- DRI appears twice** (2015 and 2016) — meaning 4 consecutive years of declining absolute CapEx spend.
- 8 streak records** identified across the dataset, spanning technology, hospitality, energy, and retail sectors.

3+ Year Rising CapEx Streaks (All Companies)

Ticker	End Year	CapEx Y- 2 (\$)	CapEx Y- 1 (\$)	CapEx Current (\$)
ADI	2016	-177,913,000	-153,960,000	-127,397,000
DRI	2015	-510,100,000	-414,800,000	-296,500,000
DRI	2016	-414,800,000	-296,500,000	-228,300,000
EA	2015	-106,000,000	-97,000,000	-95,000,000
FAST	2015	-206,540,000	-189,474,000	-155,168,000
FSLR	2014	-379,228,000	-282,576,000	-257,549,000
HST	2014	-350,000,000	-326,000,000	-306,000,000
MAC	2014	-1,204,061,000	-674,921,000	-200,645,000



Top 5 Years by Total Liabilities & Peak Operating Cash Flow per Company

Q7 — Top 5 Years by Total Liabilities (All Companies)

Sums total liabilities across all S&P 500 companies per fiscal year. WHERE IS NOT NULL excludes missing rows that could under-report yearly totals. LIMIT 5 with ORDER BY DESC returns top-N years without a subquery.

```
SELECT
  for_year,
  SUM(total_liabilities) AS total_liabilities_all
FROM fundamentals
WHERE total_liabilities IS NOT NULL
  AND for_year BETWEEN 2012 AND 2016
GROUP BY for_year
ORDER BY total_liabilities_all DESC
LIMIT 5;
```

Rank	Fiscal Year	Total Liabilities (USD)
1	2014	\$233,776,404,000
2	2013	\$233,038,615,000
3	2015	\$187,644,437,000
4	2012	\$159,250,123,000
5	2016	\$38,629,495,000

- **2014 peaks at \$233.8B** — maximum aggregate leverage across the S&P 500 dataset.
- **2016 drops sharply to \$38.6B** — only partial-year reporters submitted data for 2016, not a true decline.

S. No.	Year	Revenue (USD)
1	2014	\$233.8 B
2	2013	\$233.0 B
3	2015	\$187.6 B
4	2012	\$159.3 B
5	2016	\$38.6 B

Q8 — Year with Maximum Operating Cash Flow per Company

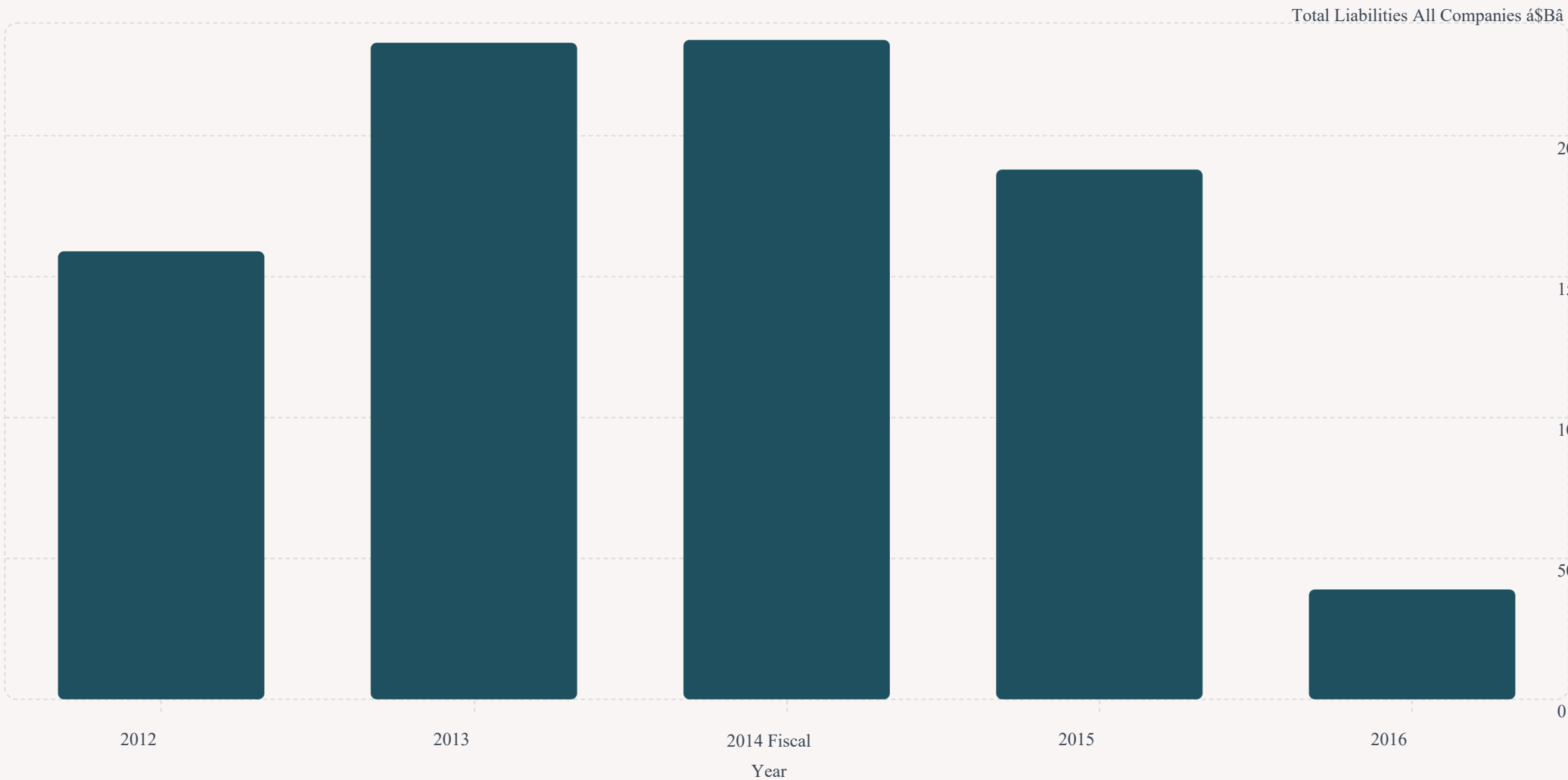
RANK() partitioned by company finds each company's peak OCF year.

RANK handles ties — two years with the same max OCF both get `rnk = 1`, unlike `ROW_NUMBER()`. CTE pattern: compute ranks first, then outer WHERE `rnk = 1` for clean filtering.

```
WITH ranked_ocf AS (
  SELECT
    ticker_symbol,
    for_year,
    net_cash_flow_operating,
    RANK() OVER (
      PARTITION BY ticker_symbol
      ORDER BY net_cash_flow_operating DESC
    ) AS rnk
  FROM fundamentals
  WHERE net_cash_flow_operating IS NOT NULL
)
SELECT
  r.ticker_symbol,
  r.for_year AS peak_ocf_year,
  ROUND(r.net_cash_flow_operating, 1) AS max_operating_cash_flow
FROM ranked_ocf r
WHERE r.rnk = 1
ORDER BY r.ticker_symbol;
```

Ticker	Peak	Max OCF (USD)
AAL	2012	\$1,285,000,000
ADI	2016	\$1,280,895,000
ADSK	2015	\$708,100,000
AJG	2013	\$372,500,000
AKAM	2015	\$764,151,000
ALB	2012	\$488,766,000
ALK	2015	\$1,584,000,000
AWK	2014	\$1,097,000,000
BCR	2013	\$1,123,300,000
AVB	2015	\$1,056,754,000

- **ALK (Alaska Air)** peaks at **\$1.58B** in 2015 — oil price drop significantly benefited airline margins.
- **RANK() vs ROW_NUMBER():** RANK preserves ties; use RANK when multiple peak years should both be flagged.



High-Equity High-Margin Companies by EPS & R&D vs Net Income Pearson Correlation

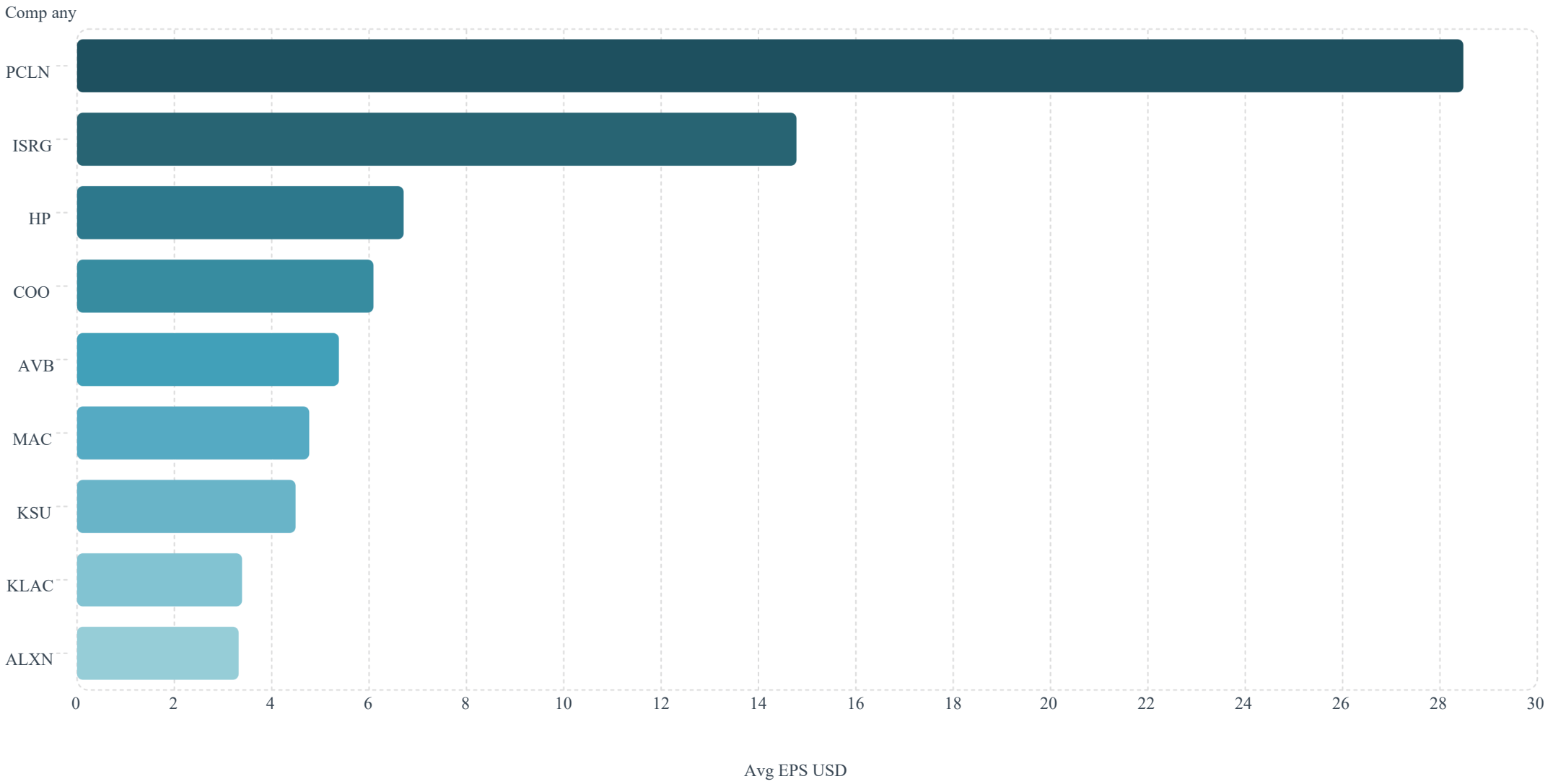
Q9 — Average EPS for High-Equity, High-Margin Companies

CROSS JOIN a single-row thresholdsCTEtobroadcastthe average equity value to every row. Filters for companies above a vg equity AND profit margin > 0, then aggregates EPS by ticker.

```
WITH thresholds AS (  
  SELECT AVG(total_equity) AS avg_equity  
  FROM fundamentals  
  WHERE total_equity IS NOT NULL  
)  
,  
filtered AS (  
  SELECT  
    f.ticker_symbol,  
    f.for_year,  
    f.earnings_per_share,  
    f.total_equity,  
    f.profit_margin  
  FROM fundamentals f  
  CROSS JOIN thresholds t  
  WHERE f.total_equity > t.avg_equity  
    AND f.profit_margin > 0  
    AND f.earnings_per_share IS NOT NULL  
)  
SELECT  
  ticker_symbol,  
  ROUND(AVG(earnings_per_share), 2) AS avg_eps,  
  ROUND(AVG(profit_margin), 2)   AS avg_margin,  
  ROUND(AVG(total_equity), 1)   AS avg_equity,  
  COUNT(*)                       AS year_count  
FROM filtered  
GROUP BY ticker_symbol  
ORDER BY avg_eps DESC;
```

Rank	Ticker	Avg EPS	Margin %	Avg Equity (\$)	Years
1	PCLN	28.48	27.00	3,896,975,000	1
2	ISRG	14.78	25.00	3,733,433,333	3
3	HP	6.71	20.50	4,667,352,000	2
4	COO	6.09	19.00	2,404,535,000	1
5	AVB	5.38	40.50	9,443,465,500	2
6	MAC	4.77	64.25	4,197,953,000	4
7	KSU	4.49	20.00	3,834,900,000	2
8	KLAC	3.39	19.50	3,575,749,000	2
9	ALXN	3.32	29.00	3,302,018,000	1
10	NFX	-1.11	57.50	3,336,500,000	2

- **PCLN leads at \$28.48 avg EPS** — asset-light travel model maximizes per-share earnings.
- **MAC shows 64.25% profit margin** across 4 qualifying years — most consistent REIT in the dataset.
- NFX negative EPS (-1.11) with 57.5% margin confirms the margin column is pre-calculated in the source data.



Thank You!

Q10 — R&D vs Net Income Pearson Correlation for AAL

Pearson correlation formula implemented in pure SQL using SUM/ POW / SQRT. Formula: $r = \text{Cov}(X,Y) / (\text{StdDev}(X) \times \text{StdDev}(Y))$, fully expressed in SQL aggregates. NULLIF(SQRT(...), 0) prevents divide-by-zero when one variable has zero variance.

```
SELECT  
  ticker_symbol,  
  COUNT(*)                       AS n,  
  ROUND(AVG(research_and_development), 1) AS avg_rd,  
  ROUND(AVG(net_income), 1)          AS avg_net_income,  
  ROUND(  
    (  
      SUM(research_and_development * net_income)  
      - COUNT(*) * AVG(research_and_development) *  
      AVG(net_income)  
    )  
    / NULLIF(  
      SQRT(  
        (SUM(POW(research_and_development, 2))  
          - COUNT(*) * POW(AVG(research_and_development), 2))  
        *  
        (SUM(POW(net_income, 2))  
          - COUNT(*) * POW(AVG(net_income), 2))  
      ), 0  
    ), 4  
  ) AS pearson_correlation  
FROM fundamentals  
WHERE ticker_symbol = 'AAL'  
  AND research_and_development IS NOT NULL  
  AND net_income IS NOT NULL  
  AND for_year BETWEEN 2012 AND 2016  
GROUP BY ticker_symbol;
```

Ticker	Observations	Avg R&D (\$)	Avg Net Income (\$)	Pearson r
AAL	1	0	-1,876,000,000	NULL

- **Result is NULL** — AAL has only 1 R&D observation; Pearson correlation requires $n \geq 2$ data points.
- AAL R&D = 0 for most years — airlines rarely report a separate R&D line item in financial statements.
- Formula fully expressed in SQL: $r = \text{Cov}(X,Y) / (\text{StdDev}(X) \times \text{StdDev}(Y))$ using SUM, POW, and SQRT aggregates.
- NULLIF(SQRT(...), 0) prevents divide-by-zero when one variable has zero variance — essential defensive SQL pattern.